上海健康医学院
医疗器械学院

学 生 实 验 报 告

课程名称：健康大数据可视化A

专业全称：数据科学与大数据技术

班　　级：22级数据科学与大数据技术2班

姓　　名：周雨晴

学　　号：B22040302020

指导教师：罗永刚

职　　称：讲师

实验日期：*2025.12.15*

上海健康醫學院
SHANGHAI UNIVERSITY OF MEDICINE & HEALTH SCIENCES

实验三 媒体可视化

实验内容	利用媒体可视化方法与技术可视化音乐数据 基于类似以下视频的例子可视化以下曲子（文件在课程资料的CASE6）

实验方法与步骤：

可视化效果 1

Sketch.js 运行代码：

```
let song;
let fft;
let input;

function setup() {
  createCanvas(windowWidth, windowHeight);
  fft = new p5.FFT();

  // 创建文件选择按钮
  input = createFileInput(handleFile);
  input.position(20, 20);

  fill(255);
  textSize(24);
  textAlign(CENTER);
}

// 处理用户选择的文件
function handleFile(file) {
  if (file.type === 'audio') {
    if (song) {
      song.stop(); // 停止上一首
    }
    // 加载用户选择的文件
    song = loadSound(file, loaded);
  } else {
    alert("请选择一个音频文件！");
  }
}

function loaded() {
```

```
  song.play();
}

function draw() {
  background(30);

  if (song && song.isPlaying()) {
    let spectrum = fft.analyze();
    noStroke();
    colorMode(HSB);
    for (let i = 0; i < spectrum.length; i+=10) { // step=10 为了减少绘制量，提升性能
      let x = map(i, 0, spectrum.length, 0, width);
      let h = -height + map(spectrum[i], 0, 255, height, 0);
      fill(i % 255, 200, 255);
      rect(x, height, width / spectrum.length * 10, h);
    }
  } else {
    text("请点击左上角按钮选择：\n13 定位难度测试 茉莉花 无伴奏合唱.flac", width/2, height/2);
  }
}
```

Index.html 代码：

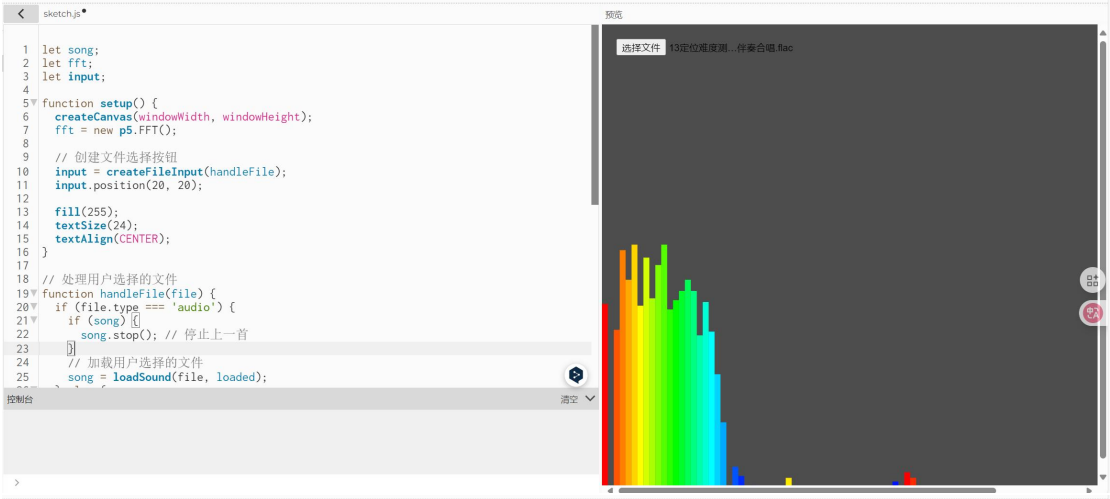
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.11/lib/p5.js">
</script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.11/lib/addons/
p5.sound.min.js"></script>


    <link rel="stylesheet" type="text/css" href="style.css">
    <meta charset="utf-8" />

  </head>
  <body>
    <main>
    </main>
    <script src="sketch.js"></script>
  </body>
```

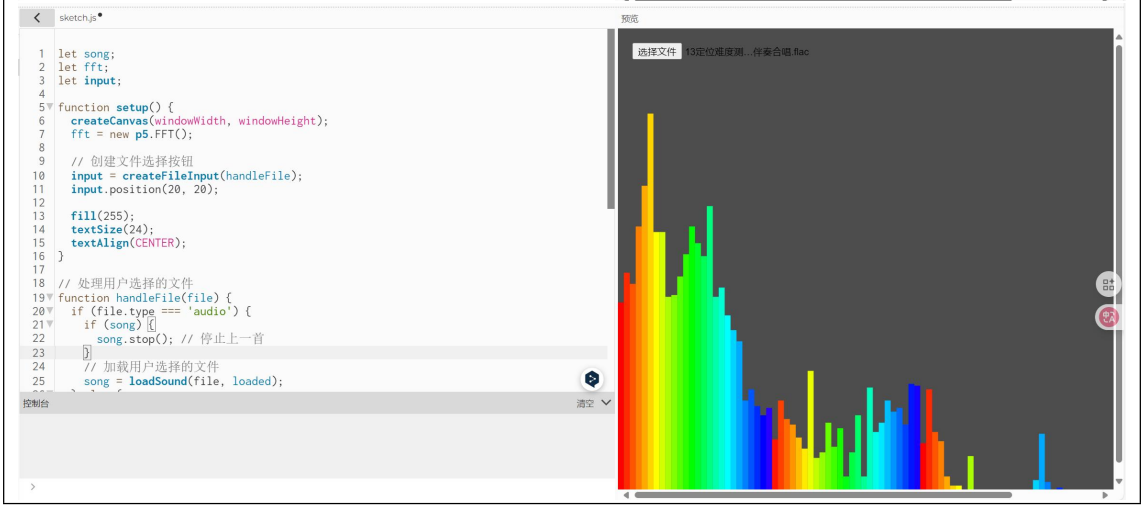
```
</html>
```

Style.css 代码：

```
html, body {
  margin: 0;
  padding: 0;
}
canvas {
  display: block;
}
```

运行结果：





可视化效果 2

Sketch.js 运行代码：

```
let song;
let fft;
let amplitude;
let input;
let isLoaded = false;

// 工业配色方案
const COLOR_BG = '#080808';        // 深邃黑背景
const COLOR_ACCENT = '#00F3FF';    // 赛博朋克青
const COLOR_DIM = '#1A2A2A';       // 暗淡的网格色
const COLOR_WHITE = '#E0E0E0';

function setup() {
  createCanvas(windowWidth, windowHeight);
  angleMode(DEGREES); // 使用角度制，方便画圆

  // 初始化音频分析工具
  fft = new p5.FFT(0.8, 512); // 平滑度 0.8，采样点 512
  amplitude = new p5.Amplitude();

  // 创建文件输入框
  input = createFileInput(handleFile);
  input.position(20, 20);
}

function handleFile(file) {
  if (file.type === 'audio') {
    if (song) { song.stop(); }
    song = loadSound(file, () => {
      isLoaded = true;
      song.loop(); // 加载完成后循环播放
    });
  } else {
    alert("请选择音频文件 (.flac, .mp3, .wav)");
  }
}

function draw() {
  background(COLOR_BG);

  // 1. 绘制背景网格 (HUD Layer)
```

```
drawGrid();

// 如果没有音乐在播放，显示待机动画或提示
if (!isLoaded || !song.isPlaying()) {
  drawStandbyText();
  return;
}

// 获取数据
let spectrum = fft.analyze(); // 频域 (0-255)
let wave = fft.waveform();    // 时域 (-1 到 1)
let level = amplitude.getLevel(); // 音量 (0 到 1)

// 移动原点到屏幕中心
translate(width / 2, height / 2);

// ----------------------------------------
// 视觉层 1: 中心核心 (Core) - 随音量缩放
// ----------------------------------------
noStroke();
fill(0, 243, 255, 50); // 半透明青色
let coreSize = map(level, 0, 1, 50, 120);
circle(0, 0, coreSize);

// 核心光环
noFill();
stroke(COLOR_ACCENT);
strokeWeight(2);
circle(0, 0, coreSize * 1.2);

// ----------------------------------------
// 视觉层 2: 环形波形 (Circular Waveform)
// ----------------------------------------
stroke(255, 100);
strokeWeight(1);
noFill();
beginShape();
for (let i = 0; i <= 360; i += 2) {
  // 将波形数组索引映射到角度
  let index = floor(map(i, 0, 360, 0, wave.length));
  // 半径基础值 + 波形振幅
  let r = 150 + map(wave[index], -1, 1, -50, 50);
  let x = r * cos(i);
  let y = r * sin(i);
```

```
    vertex(x, y);
  }
  endShape(CLOSE);

  // ----------------------------------------
  // 视觉层 3: 频谱放射线 (Frequency Bars)
  // ----------------------------------------
  stroke(COLOR_ACCENT);
  strokeWeight(2);

  // 这里的 i 代表频率，低频在数组前面，高频在后面
  // 我们只取前 180 个点，对应 0-180 度，然后镜像生成一个整圆
  for (let i = 0; i < 180; i+=2) {
    let amp = spectrum[i];
      let angle = map(i, 0, 180, -90, 90); // 将频谱映射到右半圆
      let rBase = 220;
      let rMax = map(amp, 0, 255, 0, 200);

      // 绘制右半边
      push();
      rotate(angle);
      line(rBase, 0, rBase + rMax, 0);
      pop();

      // 绘制左半边 (镜像)
      push();
      rotate(angle + 180);
      line(rBase, 0, rBase + rMax, 0);
      pop();
    }

    // ----------------------------------------
    // 视觉层 4: 旋转外圈装饰 (Rotating Ring)
    // ----------------------------------------
    rotate(frameCount * 0.2); // 缓慢自转
    stroke(COLOR_DIM);
    strokeWeight(1);
    drawingContext.setLineDash([10, 20]); // 虚线效果
    circle(0, 0, 500);
    drawingContext.setLineDash([]); // 重置虚线
  }

  // 辅助函数：绘制背景网格
  function drawGrid() {
```

```
  stroke(20, 30, 30);
  strokeWeight(1);

  // 绘制十字准星
  line(width/2, 0, width/2, height);
  line(0, height/2, width, height/2);

  // 绘制同心圆刻度
  noFill();
  circle(width/2, height/2, 440);
  circle(width/2, height/2, 800);

  // 左上角信息面板
  noStroke();
  fill(COLOR_ACCENT);
  textSize(14);
  textAlign(LEFT);
  text("SYSTEM: ONLINE", 40, 60);

  if (isLoaded) {
    text("TRACK: JASMIN FLAC", 40, 80);
    text("STATUS: PLAYING", 40, 100);
    let time = song.currentTime().toFixed(2);
    text("TIME: " + time + "s", 40, 120);
  } else {
    text("STATUS: WAITING FOR INPUT", 40, 80);
  }
}

function drawStandbyText() {
  fill(255);
  textAlign(CENTER);
  textSize(16);
  text("请点击左上角按钮选择本地 FLAC 文件", width/2, height/2);
  text("等待数据输入...", width/2, height/2 + 30);
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}

function mousePressed() {
  // 点击屏幕中心区域切换播放/暂停
  if (isLoaded) {
```

```
    let d = dist(mouseX, mouseY, width/2, height/2);
    if (d < 100) {
      if (song.isPlaying()) song.pause();
      else song.play();
    }
  }
}
```

Index.html 代码：

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.11/lib/p5.js">
</script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.11/lib/addons/
p5.sound.min.js"></script>


    <link rel="stylesheet" type="text/css" href="style.css">
    <meta charset="utf-8" />

  </head>
  <body>
    <main>
    </main>
    <script src="sketch.js"></script>
  </body>
</html>
```

Style.css 代码：

```
html, body {
  margin: 0;
  padding: 0;
}
canvas {
  display: block;
}
```

运行结果：

自动刷新 Lithe limpet p5.js 1.11.11

sketch.js

```
// 获取数据
let spectrum = fft.analyze(); // 频域 (0-255)
let wave = fft.waveform();    // 时域 (-1 到 1)
let level = amplitude.getLevel(); // 音量 (0 到 1)

// 移动原点到屏幕中心
translate(width / 2, height / 2);

// ----------------------------------------
// 视觉层 1: 中心核心 (Core) - 随音量缩放
// ----------------------------------------
noStroke();
fill(0, 243, 255, 50); // 半透明青色
let coreSize = map(level, 0, 1, 50, 120);
circle(0, 0, coreSize);

// 核心光环
noFill();
stroke(COLOR_ACCENT);
strokeWeight(2);
circle(0, 0, coreSize * 1.2);
```

控制台 清空

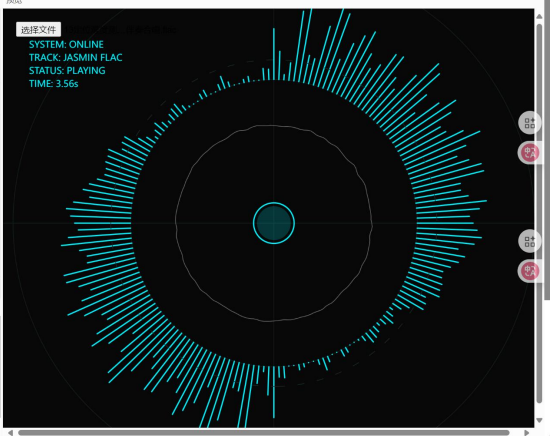
p5.js 说：您使用了p5.js的保留函数"amp"，请确保将函数名更改为其他名称。

+ 更多信息：https://p5js.org/reference/p5/amp

p5.js 说：[sketch.js，第83行] map() 期望的是Number类型的第一参数，而接收到了一个空变量。如果这不是故意的，通常是作用域的问题。

+ 更多信息：https://p5js.org/examples/data-variable-scope.html (https://p5js.org/reference/p5/map)

预览



sketch.js

```
// 获取数据
let spectrum = fft.analyze(); // 频域 (0-255)
let wave = fft.waveform();    // 时域 (-1 到 1)
let level = amplitude.getLevel(); // 音量 (0 到 1)

// 移动原点到屏幕中心
translate(width / 2, height / 2);

// ----------------------------------------
// 视觉层 1: 中心核心 (Core) - 随音量缩放
// ----------------------------------------
noStroke();
fill(0, 243, 255, 50); // 半透明青色
let coreSize = map(level, 0, 1, 50, 120);
circle(0, 0, coreSize);

// 核心光环
noFill();
stroke(COLOR_ACCENT);
strokeWeight(2);
circle(0, 0, coreSize * 1.2);
```

控制台 清空

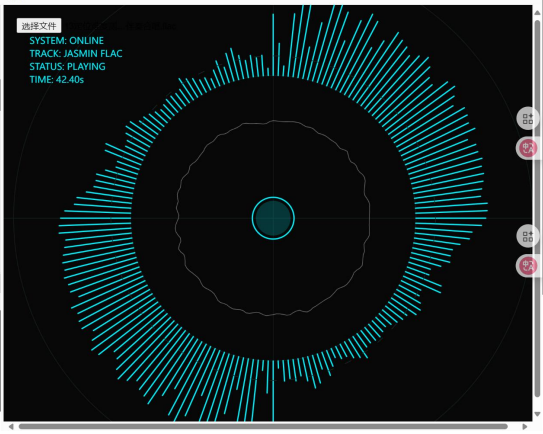
p5.js 说：您使用了p5.js的保留函数"amp"，请确保将函数名更改为其他名称。

+ 更多信息：https://p5js.org/reference/p5/amp

p5.js 说：[sketch.js，第83行] map() 期望的是Number类型的第一参数，而接收到了一个空变量。如果这不是故意的，通常是作用域的问题。

+ 更多信息：https://p5js.org/examples/data-variable-scope.html (https://p5js.org/reference/p5/map)

预览



可视化效果 3

Sketch.js 代码：

```
let song;
let fft;
let amplitude;
let particles = []; // 粒子数组

// 配色盘
const COLORS = [
  '#FFB7C5', // 樱花粉
  '#E0BBE4', // 薰衣草紫
  '#957DAD', // 柔紫
  '#D291BC', // 兰花紫
  '#FEC8D8'  // 桃色
];

function setup() {
  console.log("  [系统] p5.js 初始化开始...");

  createCanvas(windowWidth, windowHeight);
  angleMode(DEGREES);

  // 使用参数：降低精度以获得圆润感
  fft = new p5.FFT(0.8, 256);
  amplitude = new p5.Amplitude();

  // --- 关键：保持控制台响应逻辑 ---
    // 注意：这里查找的 ID 是 'audio-upload'，请确保你的 index.html 中 input 的 ID 是这个
  const fileInput = select('#audio-upload');

  if (fileInput) {
    console.log("✅ [UI] 文件上传按钮已找到，正在绑定事件...");
    fileInput.changed(handleFile);
  } else {
    // 尝试查找 file-upload 以防使用了旧的 HTML
    const altInput = select('#file-upload');
    if (altInput) {
        console.log("✅ [UI] 发现 #file-upload 按钮，正在绑定...");
        altInput.changed(handleFile);
    } else {
            console.error("❌ [错误] 找不到文件输入框！请检查 HTML id (audio-upload 或 file-upload)");
```

```
    }
  }

  textFont('M PLUS Rounded 1c');
  console.log("✅ [系统] Setup 完成，等待用户交互...");
}

// --- 关键：保持稳健的文件处理与日志逻辑 ---
function handleFile(event) {
  console.log("📂 [操作] 用户触发了文件选择...");

  // 修复 p5.js select() 获取原生文件的兼容性问题
  let files = this.elt.files || (event.target && event.target.files);

  if (files && files[0]) {
    const file = files[0];
    console.log(`📄 [文件详情] 文件名: ${file.name}`);
    console.log(`📄 [文件详情] 类型: ${file.type}`);
    console.log(`📄 [文件详情] 大小: ${(file.size / 1024 / 1024).toFixed(2)} MB`);

    if (file.type.startsWith('audio') || file.name.endsWith('.flac')) {
      console.log("🔍 [验证] 文件类型验证通过，准备加载音频...");

      if (song) {
        console.log("🗑 [播放器] 停止上一首歌曲...");
        song.stop();
      }

      // 创建 Blob URL
      let url = URL.createObjectURL(file);
      console.log(`🔗 [加载] 生成 Blob URL: ${url}`);

      song = loadSound(
        url,
        () => {
          console.log("✅ [加载] 音频加载成功！");
          console.log("▶️ [播放器] 开始循环播放...");
          song.loop();
        },
        (err) => {
          console.error("❌ [错误] 音频加载失败:", err);
```

```
        }
      );

    } else {
      console.warn("  [警告] 选择的文件似乎不是音频文件！");
      alert("请选择正确的音频文件");
    }
  } else {
    console.warn("  [操作] 用户取消了文件选择或未选中文件。");
  }
}

function draw() {
  // 1. 绘制梦幻渐变背景 (来自你的代码)
  setGradient(0, 0, width, height, color(40, 30, 70), color(20, 10, 30));

  // 2. 粒子系统更新与绘制 (背景层)
  handleParticles();

    // 如果没有音乐播放，显示待机画面
    if (!song || !song.isLoaded() || !song.isPlaying()) {
      drawStandby();
      return;
    }

    // --- 可视化核心逻辑 ---
    let spectrum = fft.analyze();
    let energy = fft.getEnergy("bass"); // 获取低音能量
    let level = amplitude.getLevel();

    translate(width / 2, height / 2);

    // 3. 绘制魔法阵 (Magic Circle)
    // 根据低音震动
    let pump = map(energy, 0, 255, 1, 1.2);
    scale(pump);

    // 外圈光晕 (Bloom Effect)
    drawingContext.shadowBlur = 30;
    drawingContext.shadowColor = 'rgba(255, 183, 197, 0.8)';

    // -------------------------
    // 魔法阵层级 A: 旋转几何
```

```
    // -------------------------
    noFill();
    strokeWeight(2);

    // 顺时针旋转的三角形
    push();
    rotate(frameCount * 0.5);
    stroke('#FFB7C5');
    drawPolygon(0, 0, 150, 3);
    pop();

    // 逆时针旋转的正方形
    push();
    rotate(-frameCount * 0.3);
    stroke('#E0BBE4');
    drawPolygon(0, 0, 130, 4);
    pop();

    // -------------------------
    // 魔法阵层级 B: 频谱光环
    // -------------------------
    // 将频谱映射为圆环
    beginShape();
    stroke(255, 200);
    strokeWeight(3);
    noFill();
    // 只取前 120 个频谱数据（低中频为主），镜像绘制
    for (let i = 0; i < 360; i+=4) {
      let index = floor(map(abs(i - 180), 0, 180, 0, 60));
      let r = 180 + map(spectrum[index], 0, 255, 0, 60);
      let x = r * cos(i);
      let y = r * sin(i);
      // 使用 curveVertex 让线条变圆润
      curveVertex(x, y);
    }
    endShape(CLOSE);

    // -------------------------
    // 魔法阵层级 C: 中心核心
    // -------------------------
    // 关闭阴影以绘制实心圆
    drawingContext.shadowBlur = 0;

    // 中心跳动的圆
```

```
  fill(255, 255, 255, 200);
  noStroke();
  let centerSize = map(level, 0, 0.5, 30, 80);
  circle(0, 0, centerSize);

  // 可以在中间绘制一个简单的符号或文字
  fill('#FF6F91');
  textSize(14);
  textAlign(CENTER, CENTER);
  text("Music", 0, 0);
}

// --- 辅助函数 ---

// 粒子系统管理
function handleParticles() {
  // 随着音乐高音产生更多粒子
  let treble = 0;
  if (song && song.isPlaying()) {
    treble = fft.getEnergy("treble");
  }
  let spawnRate = map(treble, 0, 255, 0.1, 0.8);

  if (random(1) < spawnRate) {
    particles.push(new Particle());
  }

  for (let i = particles.length - 1; i >= 0; i--) {
    particles[i].update();
    particles[i].draw();
    if (particles[i].isDead()) {
      particles.splice(i, 1);
    }
  }
}

// 粒子类
class Particle {
  constructor() {
    this.pos = createVector(random(width), height + 10);
    this.vel = createVector(random(-1, 1), random(-1, -3));
    this.size = random(3, 8);
    this.color = color(random(COLORS));
    this.alpha = 255;
```

```
      this.life = 0;
    }

    update() {
      this.pos.add(this.vel);
      this.alpha -= 1.5;
      this.life += 5;
    }

    draw() {
      noStroke();
      fill(red(this.color), green(this.color), blue(this.color), this.alpha);
      push();
      translate(this.pos.x, this.pos.y);
      rotate(this.life);
      if (random(1) > 0.5) circle(0,0, this.size);
      else rect(0,0, this.size, this.size, 1);
      pop();
    }

    isDead() {
      return this.alpha < 0;
    }
  }

  // 绘制多边形
  function drawPolygon(x, y, radius, npoints) {
    let angle = 360 / npoints;
    beginShape();
    for (let a = 0; a < 360; a += angle) {
      let sx = x + cos(a) * radius;
      let sy = y + sin(a) * radius;
      vertex(sx, sy);
    }
    endShape(CLOSE);
  }

  // 背景渐变
  function setGradient(x, y, w, h, c1, c2) {
    noFill();
    background(c1);
    drawingContext.save();
    let gradient = drawingContext.createRadialGradient(w/2, h, 0, w
```

```
/2, h, h);
  gradient.addColorStop(0, c2.toString());
  gradient.addColorStop(1, 'rgba(0,0,0,0)');
  drawingContext.fillStyle = gradient;
  drawingContext.fillRect(0, 0, w, h);
  drawingContext.restore();
}

// 待机画面
function drawStandby() {
  translate(width/2, height/2);
  fill(255);
  textAlign(CENTER);
  textSize(24);

  let scaleFactor = 1 + sin(frameCount * 2) * 0.05;
  scale(scaleFactor);

  text("waiting for music...", 0, -20);
  textSize(16);
  text("请点击左上角按钮选择音频", 0, 20);

  noFill();
  stroke(255, 100);
  circle(0, 0, 200);
}

function windowResized() {
  console.log("  [系统] 窗口大小改变");
  resizeCanvas(windowWidth, windowHeight);
}
```

Index.html 代码：

```
<!DOCTYPE html>
<html lang="zh">
  <head>
    <!-- 保持原有的 p5 库引入 -->
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.11/lib/p5.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.11/lib/addons/p5.sound.min.js"></script>

    <!-- 【新增】引入圆润的动漫风格字体 (M PLUS Rounded 1c) -->
    <link href="https://fonts.googleapis.com/css2?family=M+PLUS+Roun
```

```html
ded+1c:wght@300;700&display=swap" rel="stylesheet">

    <link rel="stylesheet" type="text/css" href="style.css">
    <meta charset="utf-8" />
    <title>Anime Music Vibe</title>
  </head>
  <body>
    <!-- 【新增】UI 控制层：为了让上传按钮好看，我们用 label 包裹隐藏的 input -->
    <div id="ui-container">
      <label for="audio-upload" class="upload-btn">
        ✨ 播放音乐 / Play Music
      </label>
      <input id="audio-upload" type="file" accept="audio/*">
    </div>

    <main>
      <!-- p5.js 会在这里自动创建 canvas -->
    </main>

    <script src="sketch.js"></script>
  </body>
</html>
```

Style.css 代码：

```css
html, body {
  margin: 0;
  padding: 0;
  background-color: #2e2a3b; /* 深紫色背景，防白屏闪烁 */
  overflow: hidden; /* 防止出现滚动条 */
  font-family: 'M PLUS Rounded 1c', sans-serif; /* 使用动漫字体 */
}

canvas {
  display: block;
}

/* UI 容器定位 */
#ui-container {
  position: absolute;
  top: 20px;
  left: 20px;
  z-index: 10; /* 保证按钮在画布上方 */
```

```
}

/* 隐藏原始丑陋的文件输入框 */
#audio-upload {
  display: none;
}

/* 自定义可爱的按钮样式 */
.upload-btn {
  display: inline-block;
  background: rgba(255, 255, 255, 0.15); /* 半透明背景 */
  border: 2px solid #ffb7c5; /* 樱花粉边框 */
  color: #ffb7c5;
  padding: 12px 24px;
  border-radius: 30px; /* 圆润的胶囊形状 */
  cursor: pointer;
  font-weight: 700;
  font-size: 14px;
  backdrop-filter: blur(5px); /* 毛玻璃效果 */
  box-shadow: 0 0 15px rgba(255, 183, 197, 0.4); /* 发光阴影 */
  transition: all 0.3s ease;
}

/* 鼠标悬停时的效果 */
.upload-btn:hover {
  background: #ffb7c5;
  color: #fff;
  transform: scale(1.05); /* 微微放大 */
  box-shadow: 0 0 25px rgba(255, 183, 197, 0.8);
}
```

运行结果：

控制台 清空

[系统] p5.js 初始化开始...

[UI] 文件上传按钮已找到，正在绑定事件...

[系统] Setup 完成，等待用户交互...

[操作] 用户触发了文件选择...

[文件详情] 文件名：13定位难度测试 茉莉花 无伴奏合唱.flac

[文件详情] 类型：audio/flac

[文件详情] 大小：22.49 MB

[验证] 文件类型验证通过，准备加载音频...

[加载] 生成 Blob URL：blob:https://preview.p5js.org/a95707b3-1c86-4a56-aad8-69e90c3fd4b3

[加载] 音频加载成功！

[播放器] 开始循环播放...

2 [系统] 窗口大小改变

项目文件

index.html

sketch.js

style.css

style.css

```
html, body {
  margin: 0;
  padding: 0;
  background-color: #2e2a3b; /* 深紫色背景，防白屏闪烁 */
  overflow: hidden; /* 防止出现滚动条 */
  font-family: 'M PLUS Rounded 1c', sans-serif; /* 使用动漫字体 */
}

canvas {
  display: block;
}

/* UI 容器定位 */
#ui-container {
  position: absolute;
  top: 20px;
  left: 20px;
  z-index: 10; /* 保证按钮在画布上方 */
}

/* 隐藏原始丑陋的文件输入框 */
#audio-upload {
  display: none;
}

/* 自定义可爱的按钮样式 */
```

控制台 清空

[UI] 文件上传按钮已找到，正在绑定事件...

[系统] Setup 完成，等待用户交互...

预览

播放音乐 / Play Music

waiting for music...

请点击左上角按钮选择音频

项目文件

index.html

sketch.js

style.css

style.css

```
html, body {
  margin: 0;
  padding: 0;
  background-color: #2e2a3b; /* 深紫色背景，防白屏闪烁 */
  overflow: hidden; /* 防止出现滚动条 */
  font-family: 'M PLUS Rounded 1c', sans-serif; /* 使用动漫字体 */
}

canvas {
  display: block;
}

/* UI 容器定位 */
#ui-container {
  position: absolute;
  top: 20px;
  left: 20px;
  z-index: 10; /* 保证按钮在画布上方 */
}

/* 隐藏原始丑陋的文件输入框 */
#audio-upload {
  display: none;
}

/* 自定义可爱的按钮样式 */
```

控制台 清空

[系统] p5.js 初始化开始...

[UI] 文件上传按钮已找到，正在绑定事件...

预览

播放音乐 / Play Music

项目文件
index.html
sketch.js
style.css
style.css
预览
html, body {
margin: 0;
padding: 0;
background-color: #2e2a3b; /* 深紫色背景，防白屏闪烁 */
overflow: hidden; /* 防止出现滚动条 */
font-family: 'M PLUS Rounded 1c', sans-serif; /* 使用动漫字体 */
}
canvas {
display: block;
}
/* UI 容器定位 */
#ui-container {
position: absolute;
top: 20px;
left: 20px;
z-index: 10; /* 保证按钮在画布上方 */
}
/* 隐藏原始丑陋的文件输入框 */
#audio-upload {
display: none;
}
/* 自定义可爱的按钮样式 */
播放音乐 / Play Music
控制台
清空
[系统] p5.js 初始化开始...
[UI] 文件上传按钮已找到，正在绑定事件...

实验总结：

通过代码实现音乐可视化（Music Visualization）是一个结合了**数字信号处理（DSP）与计算机图形学（CG）**的跨学科领域。其核心在于将听觉信号转化为视觉信号，让画面随音乐的频率、节奏、音量等参数实时变化。

一、 核心工作流 (The Pipeline)

无论使用哪种语言或工具，音乐可视化的基本流程都是一致的：

1. 音频输入 (Audio Input)：加载音频文件（mp3/wav）或捕获麦克风/系统实时音频流。
2. 音频分析 (Audio Analysis)：核心步骤。将原始的声波数据转化为计算机可理解的数值（频率、振幅）。
3. 数据处理 (Data Processing)：对原始数据进行平滑、归一化或提取特征（如 BPM、强拍）。
4. 参数映射 (Mapping)：将处理后的数值映射到视觉属性上（颜色、大小、位置、粒子速度）。
5. 渲染 (Rendering)：在屏幕上绘制每一帧画面（2D Canvas, WebGL, OpenGL 等）。

二、 核心算法与数学原理

要通过代码控制视觉，必须理解如何从音频中提取数据。

1. 时域分析 (Time Domain) - 波形 (Waveform)

原理：直接读取音频的振幅随时间的变化。

获取数据：通常是 -1.0 到 1.0 之间的浮点数数组。

视觉效果：示波器效果（一条跳动的线）、模拟心电图。适合表现声音的“质感”和瞬时响度。

2. 频域分析 (Frequency Domain) - 频谱 (Spectrum)

原理：这是可视化的灵魂。使用 快速傅里叶变换 (FFT, Fast Fourier Transform) 将时域信号转换为频域信号。它能把复杂的声音“拆解”成不同的频率成分（低音、中音、高音）。

数据结构：得到一个数组（如长度为 512 或 1024），数组索引代表频率（低->高），数值代表该频率的音量。

应用：

低频 (20-150Hz)：控制背景的震动、大物体的缩放（Kick/Bass）。

中频 (150-4000Hz)：控制主要视觉元素的变化（人声/乐器）。

高频 (4000Hz+)：控制粒子闪烁、细节纹理（镲片/空气感）。

3. 节拍检测 (Beat Detection)

原理：检测音频能量的突增（Onset Detection）。通常计算低频段的能量平均值，当瞬时能量超过平均值一定阈值时，判定为一次“Beat”。

视觉效果：屏幕闪烁、物体瞬间切换、场景变换。

三、 主流技术栈与工具

根据应用场景（Web、桌面、高性能、AI），技术选择有所不同。

1. Web 前端 (最流行、易分享)

核心 API：Web Audio API (浏览器原生，用于 FFT 和数据分析)。

绘图库：

Canvas API (2D)：适合绘制简单的柱状图、波形线。
Three.js / Babylon.js (3D)：适合构建 3D 场景，让几何体随音乐跳动。
p5.js / Tone.js：专为创意编程设计，封装极好，几行代码即可实现可视化。
D3.js：如果想做数据可视化风格的圆环或图表。

2. 创意编程与桌面应用 (高性能)
Processing (Java)：老牌创意编程工具，拥有 Minim 或 Sound 库，适合初学者学习算法。
openFrameworks (C++)：性能极强，适合大型互动装置，常配合 OpenGL 使用。
TouchDesigner (节点式编程)：虽然是可视化编程，但支持 Python 和 GLSL 代码注入，是目前专业 VJ 和新媒体艺术家的首选。

3. 游戏引擎 (沉浸式体验)
Unity (C#)：使用 AudioSource.GetSpectrumData 获取频谱。配合 VFX Graph（粒子系统）和 Shader Graph 可以制作 3A 级的可视化效果。
Unreal Engine (C++ / Blueprints)：使用 Audio Synesthesia 插件，结合 Niagara 粒子系统，效果极其华丽。

4. Python (数据科学与离线生成)
Librosa：音频分析的神器。能提取极高精度的特征（如色度图 Chroma、梅尔倒谱系数 MFCC）。
NumPy + Matplotlib：用于生成静态频谱图或科学风格的动画。
Manim：数学动画引擎，可以用来制作高质量的波形动画。

四、 视觉表现手法 (从简单到复杂)
代码写好了，画面画什么？以下是常见的映射策略：
1. 基础几何变换 (Geometric Transformation)
缩放 (Scale)：低音越重，球体越大。
颜色 (Color)：根据频率映射色相（Hue）。例如低音=红色，高音=蓝色。
位置 (Position)：波形数据直接作为 Y 轴坐标（示波器）。

2. 频谱柱状图 (Frequency Bars)
经典 Winamp 风格。将 FFT 数组绘制成一排矩形。
进阶：
环形频谱：将柱状图沿圆形排列（常用于 EDM 视频）。
镜像对称：左右对称，更具美感。

3. 粒子系统 (Particle Systems)
生成成千上万个粒子。
流场 (Flow Fields)：粒子的运动速度或方向受音频“光流”控制。
发射器：当检测到 Beat 时，发射一波粒子爆炸。

4. Shader 着色器 (GLSL)
这是高阶玩法的必经之路。在 GPU 上直接操作像素。
SDF (Signed Distance Functions)：让 3D 物体表面随音乐扭曲（Raymarching）。

纹理位移：将 FFT 数据写入一张纹理图片，在 Shader 中读取该纹理来控制平面的凹凸。

5. 反应扩散与反馈循环 (Reaction-Diffusion & Feedback)
利用上一帧的画面叠加到当前帧，配合缩放和旋转，产生迷幻的拖尾和分形效果。

五、 关键代码技巧与优化
在写代码时，直接使用原始数据通常效果很差，需要以下技巧：

- 平滑处理 (Smoothing / Lerp)：
  FFT 数据每帧变化剧烈，画面会闪烁抖动。
  解决方案：使用线性插值（Lerp）。
  代码逻辑：currentHeight = lerp(previousHeight, targetHeight, 0.1)。让变化有“阻尼感”。

- 对数刻度 (Logarithmic Scale)：
  人耳对频率的感知是对数的（低频段虽然只有 20-200Hz，但在听感上占据很大比重）。
  解决方案：不要线性地画 FFT 数组，而应该将低频部分的采样条数加宽，高频部分压缩。

- 归一化 (Normalization)：
  不同歌曲音量不同。
  解决方案：动态计算当前的平均音量，调整缩放倍数，防止画面过曝或不动。

- 音频与帧率同步：
  音频采样率通常是 44100Hz，屏幕刷新率是 60Hz。
  Web 端：使用 requestAnimationFrame 循环调用 FFT 分析方法。

一、核心工作流
音频输入：加载文件/捕获实时流
音频分析：声波→频率/振幅数值
数据处理：平滑、归一化、特征提取
参数映射：数值→视觉属性（颜色/大小等）
渲染：2D Canvas/WebGL等绘制帧画面
二、核心算法与数学原理
时域分析：波形（振幅-时间）→示波器/心电图效果
频域分析：FFT变换→频谱（低频/中频/高频）
节拍检测：能量突增判定→闪烁/场景切换
三、主流技术栈与工具
Web前端：Web Audio API+Canvas/Three.js/p5.js
创意编程/桌面：Processing/openFrameworks/TouchDesigner
游戏引擎：Unity(C#)/Unreal Engine(C++/Blueprints)
Python：Librosa+NumPy/Matplotlib/Manim
四、视觉表现手法
基础几何变换：缩放/颜色/位置映射
频谱柱状图：经典/环形/镜像对称
粒子系统：流场控制/Beat触发发射
Shader着色器：SDF/纹理位移（GPU像素操作）
反应扩散与反馈循环：帧叠加→迷幻拖尾/分形
五、关键代码技巧与优化
平滑处理：Lerp插值减少画面抖动
对数刻度：适配人耳频率感知
归一化：动态调整音量缩放
音频与帧率同步：requestAnimationFrame（Web端）
六、总结（实践方向）
入门级：Canvas+Web Audio API→跳动圆形
进阶级：Three.js→3D隧道（鼓点加速）
专家级：GLSL→分形着色器（音频控制迭代/颜色）
前沿级：AI生成→Stable Diffusion/TouchDesigner联动